



Universidad Autónoma de Querétaro
Facultad De Ingeniería



**FACULTAD DE
INGENIERÍA**

Problema del Vendedor Viajero (TSP)

Algoritmos Metaheurísticos Tarea 3

P R E S E N T A

Ing. Daniel Eduardo González Alvarado

Profesor

Dr. Marco Antonio Aceves Fernández

03 de Octubre de 2025

Índice

1. Introducción	1
1.1. Cruza PMX y PBX	1
1.1.1. PMX (Partially Mapped Crossover)	1
1.1.2. Funcionamiento del PMX	2
1.1.3. Ejemplo de PMX	2
1.1.4. PBX (Position-Based Crossover)	2
1.1.5. Funcionamiento del PBX	2
1.1.6. Ejemplo de PBX	2
1.2. Métodos de Mutación en el TSP	3
1.2.1. Mutación por Barajado (Scramble Mutation)	3
1.2.2. Mutación por Transposición de Bloques (Block Transpose Mutation)	3
1.3. Ciudades	3
1.4. Cálculo de Distancias (Haversine)	4
2. Desarrollo	4
2.1. Codificación	5
2.2. Generar Población Inicial	6
2.3. Evaluar Aptitud	6
2.4. Ordenar Población	7
2.5. Selección	7
2.6. Cruce	8
2.6.1. PMX (Partially Mapped Crossover)	8
2.6.2. PBX (Position-Based Crossover)	9
2.7. Mutación	9
2.7.1. Mutación de Mezcla (Scramble Mutation)	9
2.7.2. Mutación de Intercambio de Bloques (Block Transpose Mutation)	10
2.8. Ejecucion del Algoritmo Genético	11
2.9. Poblacion Inicial: 100	12
2.9.1. Cruce PMX	12
2.9.2. Cruce PBX	13
2.10. Poblacion Inicial: 200	14
2.10.1. Cruce PMX	14
2.10.2. Cruce PBX	15
2.11. Poblacion Inicial: 500	16
2.11.1. Cruce PMX	16
2.11.2. Cruce PBX	17
2.12. Poblacion Inicial: 1000	18
2.12.1. Cruce PMX	18
2.12.2. Cruce PBX	19
3. Resultados	20
4. Discusión	21
5. Conclusión	22

1. Introducción

El Problema del vendedor viajero o Traveling Salesman Problem (TSP por sus siglas en ingles) es uno de los problemas más estudiados en el campo de la optimización combinatoria. Consiste en encontrar el recorrido más corto posible que permite a un viajero visitar un conjunto N de ciudades exactamente una vez y regresar a la ciudad de origen. A pesar de su simplicidad en la formulación, el TSP es un problema *NP-hard*, lo que significa que no se conoce un algoritmo eficiente para resolverlo en tiempo polinómico para todos los casos.[1]

Es decir, encontrar una permutación:

$P = \{C_0, C_1, \dots, C_N - 1\}$ tal que $d_p = \sum_{i=0}^{N-1} d(c_i, c_{i+1})$ sea mínima.[2]

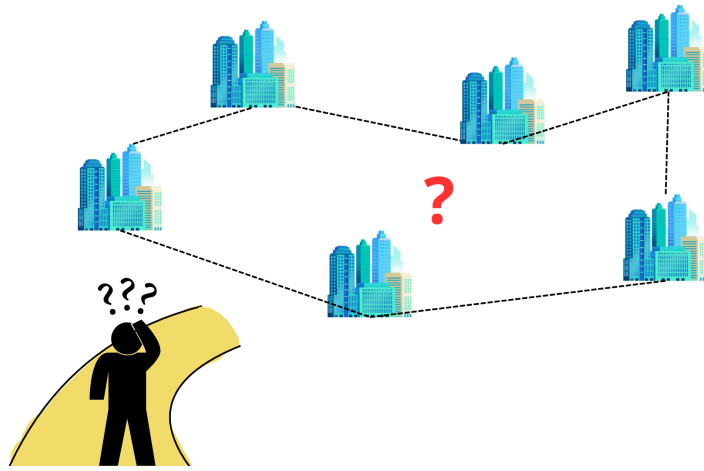


Figura 1: Planteamiento del problema del vendedor viajero (TSP).

Este problema tiene múltiples aplicaciones prácticas en áreas como la logística, la planificación de rutas, la fabricación y la biología computacional, entre otras. Debido a su importancia teórica y práctica, ha motivado el desarrollo de diversos métodos de solución, desde algoritmos exactos basados en técnicas de ramificación y poda hasta heurísticas y metaheurísticas capaces de ofrecer buenas soluciones en tiempos razonables para instancias de gran tamaño.

Para esta ocasión, aplicaremos algoritmos genéticos para resolver este problema. Ya se ha explorado lo que son los algoritmos genéticos, su funcionamiento y sus aplicaciones en problemas de optimización por lo que lo abordaremos de manera más general en el contexto del TSP.

En este caso usaremos los métodos de cruce PMX y PBX, y las mutaciones de mezcla (scramble) y de intercambio de bloques (block transpose).

1.1. Cruza PMX y PBX

1.1.1. PMX (Partially Mapped Crossover)

El **PMX** o Cruce Parcialmente Mapeado es uno de los operadores de cruce más utilizados para el TSP. Su principio fundamental es mantener el arreglo relativo de los genes de un padre mientras introduce variación del otro.

1.1.2. Funcionamiento del PMX

1. **Selección de puntos de corte:** Se seleccionan aleatoriamente dos puntos de corte que dividen los cromosomas de los padres en tres segmentos (izquierdo, medio y derecho).
2. **Copia del segmento medio:** El segmento medio del primer padre se copia directamente al descendiente, preservando su orden exacto.
3. **Establecimiento de relaciones de mapeo:** Se crea una relación de mapeo entre los elementos del segmento medio de ambos padres para resolver conflictos.
4. **Legalización del descendiente:** Se utiliza el mapeo para completar las posiciones restantes, asegurando que no haya duplicaciones y que sea una permutación válida.

1.1.3. Ejemplo de PMX

Consideremos los siguientes padres:

$$\text{Padre 1} = [1, 2, 3, 4, 5, 6, 7, 8, 9] \quad (1)$$

$$\text{Padre 2} = [5, 4, 6, 9, 2, 1, 7, 8, 3] \quad (2)$$

Con puntos de corte en las posiciones 2 y 6, el segmento medio del Padre 1 es $[3, 4, 5, 6]$. Este segmento se copia al descendiente y se establece un mapeo con el segmento correspondiente del Padre 2 para completar las posiciones restantes sin duplicaciones.

1.1.4. PBX (Position-Based Crossover)

El **PBX** o Cruce Basado en Posición selecciona un subconjunto de posiciones del primer padre y preserva el orden relativo de los elementos restantes del segundo padre.

1.1.5. Funcionamiento del PBX

1. **Selección aleatoria de posiciones:** Se seleccionan aleatoriamente varias posiciones (no necesariamente consecutivas) del primer padre.
2. **Copia de elementos seleccionados:** Los elementos que se encuentran en las posiciones seleccionadas se copian directamente al descendiente en las mismas posiciones.
3. **Eliminación de duplicados:** Se eliminan del segundo padre todos los elementos que ya fueron copiados.
4. **Completado del descendiente:** Las posiciones restantes se llenan con los elementos sobrantes del segundo padre, manteniendo su orden relativo original.

1.1.6. Ejemplo de PBX

Utilizando los mismos padres:

$$\text{Padre 1} = [1, 2, 3, 4, 5, 6, 7, 8, 9] \quad (3)$$

$$\text{Padre 2} = [5, 4, 6, 9, 2, 1, 7, 8, 3] \quad (4)$$

Si se seleccionan las posiciones 3, 5 y 6 (correspondientes a los elementos 3, 5 y 6 del Padre 1) estos elementos se preservan en sus posiciones originales. Las posiciones restantes se completan con los elementos del Padre 2 que no han sido utilizados, manteniendo su orden relativo.

1.2. Métodos de Mutación en el TSP

En los algoritmos genéticos, la mutación es un operador que introduce diversidad en la población al modificar las rutas de manera controlada. En este caso se decidieron utilizar los siguientes métodos de mutación debido a su efectividad en problemas de permutación como el TSP.:

1.2.1. Mutación por Barajado (Scramble Mutation)

Este método consiste en seleccionar un segmento de la ruta y reordenar de manera aleatoria las ciudades dentro de este segmento. De esta forma, la estructura global de la ruta se mantiene, pero el orden local de algunas ciudades cambia, lo que puede llevar a descubrir nuevas combinaciones con mejor aptitud.

Ejemplo:

$$[1, 2, 3, 4, 5, 6, 7, 8] \xrightarrow{\text{Scramble en } [3,4,5,6]} [1, 2, 5, 3, 6, 4, 7, 8]$$

1.2.2. Mutación por Transposición de Bloques (Block Transpose Mutation)

En este método se eligen dos bloques de la ruta y se intercambian de lugar de manera directa. Esto modifica la posición relativa de grupos completos de ciudades, lo que nos permite explorar nuevas configuraciones manteniendo intacto el orden dentro de cada bloque.

Ejemplo:

$$[1, 2, 3, 4, 5, 6, 7, 8] \xrightarrow{\text{Intercambiar } [2,3] \text{ con } [6,7]} [1, 6, 7, 4, 5, 2, 3, 8]$$

Dicho de manera más sencilla *scramble* altera el orden interno de un subconjunto de ciudades, mientras que *block transpose* cambia la posición relativa de segmentos completos de la ruta.

1.3. Ciudades

Para este ejercicio se utilizaron las siguientes ciudades con su etiqueta correspondiente:

- | | | |
|--------------------|-----------------------|---------------------|
| ■ 0 — Mexico City | ■ 6 — Merida | ■ 12 — Buenos Aires |
| ■ 1 — Quito | ■ 7 — Washington D.C. | ■ 13 — New York |
| ■ 2 — Miami | ■ 8 — Monterrey | ■ 14 — Panama City |
| ■ 3 — San Salvador | ■ 9 — Managua | ■ 15 — Brasilia |
| ■ 4 — Mendoza | ■ 10 — Caracas | ■ 16 — Montevideo |
| ■ 5 — Guadalajara | ■ 11 — Boston | ■ 17 — Bogota |



Figura 2: Ciudades consideradas en el problema del vendedor viajero con su etiqueta.

1.4. Cálculo de Distancias (Haversine)

Es importante mencionar que las coordenadas tomadas para este ejercicio fueron coordenadas geodesicas (latitud y longitud) y no coordenadas planas, debido a que el uso de coordenadas planas puede introducir errores significativos en la estimación de distancias reales entre ciudades, dicho de otra manera podríamos estar tomando atajos que no existen en la realidad (Cortando la tierra por decirlo de una manera). Debido a esto la distancia entre dos ciudades se calculó usando la fórmula del haversine. Esta fórmula considera la curvatura de la Tierra y proporciona una estimación más precisa de la distancia entre dos puntos geográficos. La fórmula del haversine es la siguiente:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right)$$

Con esto creamos una matriz de distancias entre todas las ciudades, que luego se utiliza para evaluar la aptitud de cada ruta en el algoritmo genético.

2. Desarrollo

Tomando en cuenta que nuestra Poblacion total son las combinaciones de las 18 ciudades tenemos que: El total seria $18! = 6,402,373,705,728,000$ posibles combinaciones, lo cual es un numero muy grande para poder evaluar todas las posibles soluciones, por lo que se opto por utilizar algoritmos geneticos para poder encontrar una solucion aproximada al problema.

2.1. Codificación

Para representar las soluciones del TSP en el contexto de algoritmos genéticos, se utiliza una codificación basada en permutaciones. Cada cromosoma es una secuencia ordenada de números, donde cada número representa una ciudad específica y su posición en la secuencia indica el orden en que se visitan las ciudades.

Usando la lista mostrada en la sección 1.3, un cromosoma podría representarse de la siguiente manera (De Norte a Sur):

$$[11, 13, 7, 2, 8, 5, 0, 6, 3, 9, 14, 10, 17, 1, 4, 12, 16, 15]$$

Esta representación asegura que cada ciudad se visite exactamente una vez, y el recorrido comienza y termina en la ciudad de origen. Las etiquetas numéricas facilitan la manipulación de las rutas durante las operaciones genéticas como la selección, el cruce y la mutación.



Figura 3: Ruta propuesta de Norte a Sur para el problema del vendedor viajero.

Esta ruta solo tiene como objeto una propuesta basada en un análisis visual de las ciudades que nos sirviera para comparar mas adelante, y no representa la mejor ruta posible, ademas de esto debemos tomar en cuenta que el punto de partida puede ser cualquiera de las ciudades, por lo que la ruta puede variar dependiendo de la ciudad inicial.

2.2. Generar Población Inicial

Para generar nuestra población inicial, se crean múltiples cromosomas (rutas) usando la función `sample` de la librería `random` de `python` esto nos crea una lista de la longitud deseada sin repetir elementos y en un orden aleatorio, asegurando que cada ciudad se visite exactamente una vez en cada ruta.



Figura 4: Funcion generadora de rutas

2.3. Evaluar Aptitud

Como se menciona en la sección 1.4, para evaluar la aptitud de cada cromosoma (ruta), se utiliza la fórmula de Haversine.

Para este problema, tenemos dos opciones: tomar la aptitud de una ruta como el inverso de la distancia total recorrida para buscar maximizar la aptitud, o simplemente tomar la distancia total como la medida a minimizar siendo este el enfoque mas comun en el TSP es decir tomar la menor aptitud como la mejor solucion. En este caso se opto por la segunda opcion.

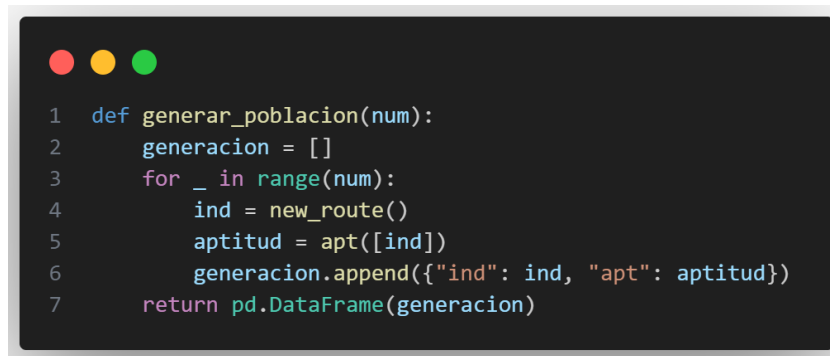
Una vez entendido esto calculamos la matriz de distancias de Haversine entre todas las ciudades y la almacenamos en una matriz para facilitar el calculo de distancias entre ciudades.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
0	0	306613.9066	632477.8441	2024666.904	3245136.361	3855874.331	3667299.526	2940971.334	3631959.727	3668908.499	3798722.935	3567181.952	4196286.274	4792674.683	6919273.623	8722912.618	8654135.824	8369476.362
1	306613.9066	0	325870.7026	1757493.983	2943690.694	3551688.717	3360687.313	2642612.713	3346101.934	3396838.634	3569009.201	3433650.879	4003266.3	4574399.09	6876403.136	8603897.315	8526515.946	8201231.966
2	632477.8441	325870.7026	0	1491668.041	2623895.564	3228432.253	3034822.612	2328556.61	3047518.413	3115566.263	3339098.916	3317286.892	3814673.858	4354202.706	6842358.746	8483303.247	8396834.19	8028305.886
3	2024666.904	1757493.983	1491668.041	0	2012518.994	2428448.437	2067095.163	1097695.333	1637663.039	1644646.064	1873347.123	2200658.044	2430237.147	2895331.275	5866509.474	7204080.759	7095185.748	6631519.357
4	3245136.361	2943690.694	2623895.564	2012518.994	0	637376.9553	706143.506	1211116.179	1766474.343	2110338.211	2905757.07	3896665.04	3639152.513	3711443.731	7406833.905	8182011.634	8027012.299	7314459.496
5	3855874.331	3551688.717	3228432.253	2428448.437	637376.9553	0	459576.2317	1427923.825	1689817.205	2056303.358	2900999.894	4052493.268	3625881.229	3562879.674	7392448.411	7934021.429	7766909.903	6992359.999
6	3667299.526	3360687.313	3034822.612	2067095.163	706143.506	459576.2317	0	1010928.491	1236525.053	1602407.487	2444577.487	3596037.685	3171796.602	3136783.025	6942713.76	7553433.915	7391650.942	6648170.185
7	2940971.334	2642612.713	2328556.61	1097695.333	1211116.179	1427923.825	1010928.491	0	809655.7626	1048434.252	1742841.28	2689435.686	2466092.539	2647515.568	6211755.524	7144522.885	7004745.442	6381747.892
8	3631959.727	3346101.934	3047518.413	1637663.039	1766474.343	1689817.205	1236525.053	809655.7626	0	366515.6256	1213386.863	2448027.541	1936085.832	1945200.546	5706405.804	6428744.623	6279549.234	5613295.536
9	3668908.499	3396838.634	3115566.263	1644646.064	2110338.211	2056303.358	1602407.487	1048434.252	366515.6256	0	847954.7354	2113909.147	1569667.541	1615255.475	5341339.543	6112009.947	5967702.878	5333317.329
10	3798722.935	3569009.201	3339098.916	1873347.123	2905757.07	2900999.894	2444577.487	1742841.28	1213386.863	847954.7354	0	1354611.987	734173.2267	1022094.879	4509570.992	5441962.989	5313867.679	4778528.958
11	3567181.952	3433650.879	3317286.892	2200658.044	3896665.04	4052493.268	3596037.685	2689435.686	2448027.541	2113909.147	1354611.987	0	1016470.462	1748878.097	3665858.556	5171217.32	5093062.734	4826465.036
12	4196286.274	4003266.3	3814673.858	2430237.147	3639152.513	3625881.229	3171796.602	2466092.539	1936085.832	1569667.541	734173.2267	1016470.462	0	737078.7085	3776129.471	4786040.591	4670292.775	4216942.952
13	4792674.683	4574399.09	4354202.706	2895331.275	3711443.731	3562879.674	3136783.025	2647515.568	1945200.546	1615255.475	1022094.879	1748878.097	737078.7085	0	3897006.278	4498418.834	4357237.284	3770269.218
14	6919273.623	6876403.136	6842358.746	5866509.474	7406833.905	7392448.411	6942713.76	6211755.524	5706405.804	5341339.543	4509570.992	3665858.556	3776129.471	3897006.278	0	2342638.495	2412341.07	2944526.389
15	8722912.618	8603897.315	8483303.247	7204080.759	8182011.634	7934021.429	7553433.915	7144522.885	6428744.623	6112009.947	5441962.989	5171217.32	4786040.591	4498418.834	2342638.495	0	204263.2769	1188830.391
16	8654135.824	8526515.946	8396834.19	7095185.748	8027012.299	7766909.903	7391650.942	7004745.442	6279349.234	5967702.878	5313867.679	5093062.734	4670292.775	4357237.284	2412341.07	204263.2769	0	984642.3331
17	8369476.362	8201231.966	8028305.886	6631519.357	7314459.496	6992359.999	6648170.185	6381747.892	5613295.536	5333317.329	4778528.958	4826465.036	4216942.952	3770269.218	2944526.389	1188830.391	984642.3331	0

Figura 5: Matriz de distancias entre ciudades usando la formula de Haversine

Como podemos observar en la figura 5 la matriz es simetrica de (18×18) y la diagonal principal es cero, ya que la distancia de una ciudad a si misma es cero. Para medir la aptitud de un cromosoma, se calcula la distancia entre cada par de ciudades consecutivas en la ruta, sumando estas distancias para obtener la distancia total del recorrido.

Con lo anterior podemos generar nuestra poblacion inicial y evaluar su aptitud con la siguiente funcion:



```

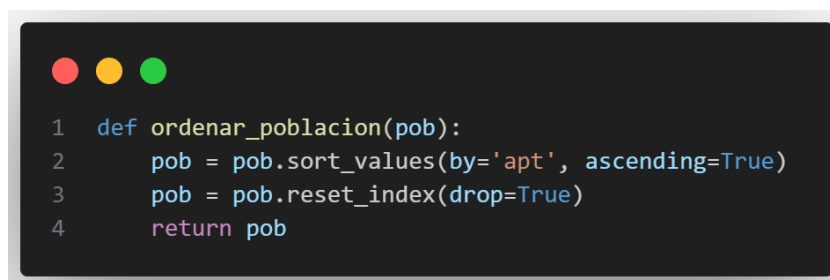
1 def generar_poblacion(num):
2     generacion = []
3     for _ in range(num):
4         ind = new_route()
5         aptitud = apt([ind])
6         generacion.append({"ind": ind, "apt": aptitud})
7     return pd.DataFrame(generacion)

```

Figura 6: Funcion generadora de la poblacion con aptitud

2.4. Ordenar Población

Para ordenar la población en función de la aptitud de cada cromosoma (ruta), se utiliza la función `sorted` de Python junto con un `reset_index` de pandas para reordenar los índices después de la ordenación tomando como clave de ordenación la columna 'apt'.



```

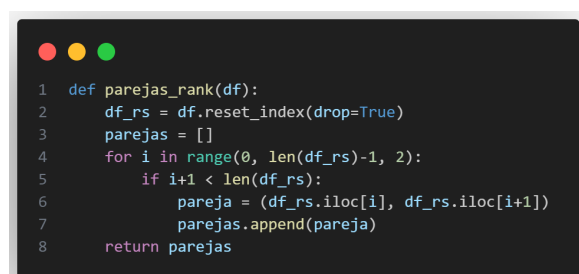
1 def ordenar_poblacion(pob):
2     pob = pob.sort_values(by='apt', ascending=True)
3     pob = pob.reset_index(drop=True)
4     return pob

```

Figura 7: Funcion para ordenar la poblacion por aptitud

2.5. Selección

Para la selección de padres en el proceso de cruce, usaremos los metodos de torneo y ranking, para lo cual en cada generacion tomaremos la mitad superior de la poblacion ordenada para aplicar rank y la mitad inferior para aplicar torneo, esto con el objetivo de mantener diversidad en la poblacion y evitar caer en un minimo local.

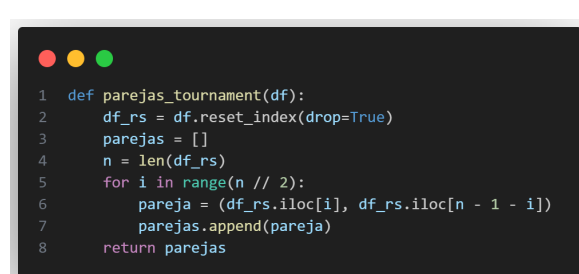


```

1 def parejas_rank(df):
2     df_rs = df.reset_index(drop=True)
3     parejas = []
4     for i in range(0, len(df_rs)-1, 2):
5         if i+1 < len(df_rs):
6             pareja = (df_rs.iloc[i], df_rs.iloc[i+1])
7             parejas.append(pareja)
8     return parejas

```

(a) Funcion ranking



```

1 def parejas_tournament(df):
2     df_rs = df.reset_index(drop=True)
3     parejas = []
4     n = len(df_rs)
5     for i in range(n // 2):
6         pareja = (df_rs.iloc[i], df_rs.iloc[n - 1 - i])
7         parejas.append(pareja)
8     return parejas

```

(b) Funcion torneo

Figura 8: Funciones de seleccion de padres

2.6. Cruce

Para el cruce de los cromosomas (rutas), se implementaron dos métodos: PMX y PBX.

2.6.1. PMX (Partially Mapped Crossover)

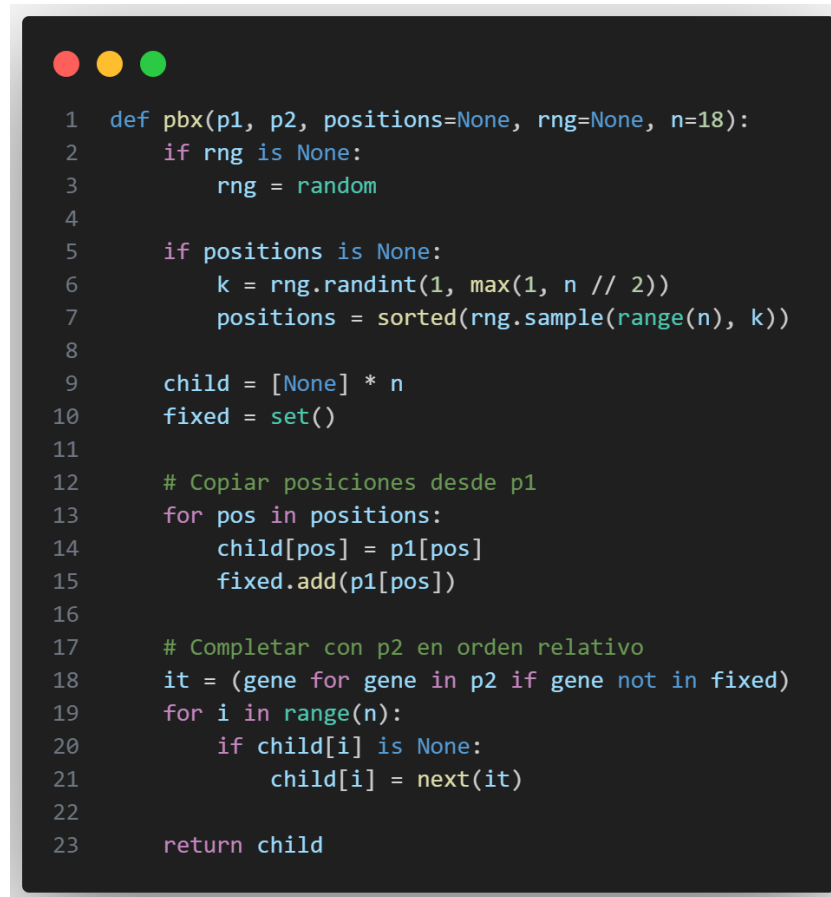
El cruce PMX selecciona dos padres y se elige un segmento de la ruta de uno de ellos. Luego, se mapea este segmento a la ruta del otro padre, asegurando que no se repitan ciudades. Como se explica en la sección 1.1.1.

```
1 def pmx(p1, p2, i=None, j=None):
2     n = len(p1)
3     assert len(p1) == len(p2) and n >= 2
4
5     if i is None or j is None:
6         i, j = sorted(random.sample(range(n), 2))
7
8     h1 = [None] * n
9     h2 = [None] * n
10
11     # Copiar segmento central
12     h1[i:j+1] = p1[i:j+1]
13     h2[i:j+1] = p2[i:j+1]
14
15     # Mapeo bidireccional
16     map12 = {}
17     for a, b in zip(p1[i:j+1], p2[i:j+1]):
18         map12[a] = b
19         map12[b] = a
20
21     seg1 = set(h1[i:j+1]) # genes fijos en h1 (de P1)
22     seg2 = set(h2[i:j+1]) # genes fijos en h2 (de P2)
23
24     # Rellenar h1 con P2 respetando el mapeo
25     for pos in list(range(0, i)) + list(range(j+1, n)):
26         gene = p2[pos]
27         count = 0
28         original_gene = gene
29         while gene in seg1:
30             gene = map12[gene]
31             count += 1
32             if count > n: # Previene ciclo infinito
33                 gene = original_gene
34                 break
35         h1[pos] = gene
36
37     # Rellenar h2 con P1 respetando el mapeo
38     for pos in list(range(0, i)) + list(range(j+1, n)):
39         gene = p1[pos]
40         count = 0
41         original_gene = gene
42         while gene in seg2:
43             gene = map12[gene]
44             count += 1
45             if count > n: # Previene ciclo infinito
46                 gene = original_gene
47                 break
48         h2[pos] = gene
49
50     universo = list(p1)
51     h1 = _repair_no_dups(h1, i, j, universo)
52     h2 = _repair_no_dups(h2, i, j, universo)
53
54     return h1, h2
```

Figura 9: Funcion de cruce PMX

2.6.2. PBX (Position-Based Crossover)

El cruce PBX selecciona aleatoriamente varias posiciones en la ruta de un padre y copia las ciudades en esas posiciones al hijo. Luego, completa las posiciones restantes con las ciudades del otro padre, manteniendo el orden relativo. Este se explica en la sección 1.1.5.

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The code is written in Python and defines a function named pbx. The function takes parameters p1, p2, positions (defaulting to None), rng (defaulting to None), and n (defaulting to 18). It includes logic to initialize rng and positions, create a child list, copy positions from p1, and then complete the child by adding cities from p2 in their relative order. The code is as follows:

```
1 def pbx(p1, p2, positions=None, rng=None, n=18):
2     if rng is None:
3         rng = random
4
5     if positions is None:
6         k = rng.randint(1, max(1, n // 2))
7         positions = sorted(rng.sample(range(n), k))
8
9     child = [None] * n
10    fixed = set()
11
12    # Copiar posiciones desde p1
13    for pos in positions:
14        child[pos] = p1[pos]
15        fixed.add(p1[pos])
16
17    # Completar con p2 en orden relativo
18    it = (gene for gene in p2 if gene not in fixed)
19    for i in range(n):
20        if child[i] is None:
21            child[i] = next(it)
22
23    return child
```

Figura 10: Funcion de cruce PBX

2.7. Mutación

Para la mutación de las rutas, se implementaron dos métodos: mezcla (scramble) e intercambio de bloques (block transpose).

2.7.1. Mutación de Mezcla (Scramble Mutation)

La mutación de mezcla selecciona un segmento aleatorio de la ruta y reordena las ciudades dentro de ese segmento de manera aleatoria.

```

1 def mut_scramble(ruta, fix_first=True, min_len=2, max_len=None):
2     """
3     Baraja aleatoriamente las ciudades dentro de un subsegmento [i:j].
4     """
5     if len(ruta) < 4:
6         return ruta[:] # nada útil que barajar
7
8     # Soporte ruta cerrada (último = primero)
9     closed = (ruta[0] == ruta[-1])
10    path = ruta[:-1] if closed else ruta[:]
11
12    n = len(path)
13    start_idx = 1 if fix_first else 0
14    usable = n - start_idx
15    if usable < 3:
16        return ruta[:]
17
18    if max_len is None:
19        max_len = usable # sin límite superior
20
21    max_len = max(min_len, min(max_len, usable))
22    l = random.randint(min_len, max_len)
23    i = random.randint(start_idx, n - l)
24    j = i + l
25
26    seg = path[i:j]
27    random.shuffle(seg)
28    new_path = path[:i] + seg + path[j:]
29
30    return new_path + [new_path[0]] if closed else new_path
31

```

Figura 11: Funcion de mutacion scramble

2.7.2. Mutación de Intercambio de Bloques (Block Transpose Mutation)

La mutación de intercambio de bloques selecciona dos segmentos de la ruta y los intercambia entre sí.

```

1 def mut_block_transpose(ruta, fix_first=True):
2     """
3     Intercambia dos bloques contiguos
4     """
5     if len(ruta) < 4:
6         return ruta[:]
7
8     closed = (ruta[0] == ruta[-1])
9     path = ruta[:-1] if closed else ruta[:]
10
11    n = len(path)
12    start_idx = 1 if fix_first else 0
13    if n - start_idx < 3:
14        return ruta[:]
15
16    # Elegir i, j, k con j-i>0 y k-j>0
17    i = random.randint(start_idx, n - 3)
18    j = random.randint(i + 1, n - 2)
19    k = random.randint(j + 1, n)
20
21    A = path[i:j]
22    B = path[j:k]
23    new_path = path[:i] + B + A + path[k:]
24
25    return new_path + [new_path[0]] if closed else new_path
26

```

Figura 12: Funcion de mutacion block transpose

2.8. Ejecucion del Algoritmo Genético

Por ultimo, se implementa la función principal del algoritmo genético que integra todos los componentes anteriores. Esta función parte de una población inicial, N generaciones y tipo de cruce ademas de ofrecernos una grafica con el progreso de la mejor solución encontrada. De manera interna se selecciona el tipo de mutacion si se estanca la poblacion.

```
1 def Algoritmo_Genetic(poblacion_inicial, num_generaciones, Cruza='PMX', Grafica=False):
2     poblacion = poblacion_inicial.copy()
3
4     df_metricas = pd.DataFrame(columns=["ind", "apt", "Avg_Gen_Fitness", "Delta", "Avg_Delta"])
5     df_metricas = metrics(poblacion, df_metricas, Clean=True)
6
7     patience = 10      # generaciones sin mejora para considerar "estancado"
8     tol = 1e-9         # tolerancia de mejora
9     stagnation = 0
10    best_so_far = float(poblacion.iloc[0]["apt"])
11    mutate_frac = 0.10  # 10% de la población
12
13    mitad1, mitad2 = dividir_mitad(poblacion)
14
15    for gen in range(num_generaciones):
16        parejas_r = parejas_rank(mitad1)
17        parejas_t = parejas_tournament(mitad2)
18
19        if Cruza == 'PBX':
20            hijos_r = cruzar_pbx_parejas(parejas_r)
21            hijos_t = cruzar_pbx_parejas(parejas_t)
22        elif Cruza == 'PMX':
23            hijos_r = cruzar_pmx_parejas(parejas_r)
24            hijos_t = cruzar_pmx_parejas(parejas_t)
25        gen_hijos = pd.DataFrame({'ind': hijos_r + hijos_t})
26
27        if stagnation >= patience:
28            k = max(1, math.ceil(len(gen_hijos) * mutate_frac))
29            idx_to_mutate = random.sample(range(len(gen_hijos)), k=min(k, len(gen_hijos)))
30            # aplicar uno de las dos mutaciones (scramble o block transpose)
31            for idx in idx_to_mutate:
32                ind = gen_hijos.at[idx, 'ind']
33                ind = mut_scramble(ind) if random.random() < 0.5 else mut_block_transpose(ind)
34                gen_hijos.at[idx, 'ind'] = ind
35
36        # evaluar hijos
37        gen_hijos['apt'] = gen_hijos['ind'].apply(apt_ind)
38
39        # integrar con población (como ya lo tenias)
40        poblacion = pd.concat([poblacion, gen_hijos], ignore_index=True)
41        poblacion = ordenar_poblacion(poblacion).iloc[:len(poblacion_inicial)].reset_index(drop=True)
42
43        # actualizar métricas
44        df_metricas = metrics(poblacion, df_metricas)
45
46        curr_best = float(poblacion.iloc[0]['apt'])
47        if (best_so_far - curr_best) > tol: # mejora (menor distancia)
48            best_so_far = curr_best
49            stagnation = 0
50        else:
51            stagnation += 1
52
53        # re-dividir para la próxima generación
54        mitad1, mitad2 = dividir_mitad(poblacion)
55
56        goat_ind = poblacion.iloc[0]['ind']
57        goat_apt = (poblacion.iloc[0]['apt'])/1000
58        print("Mejor ruta: ", goat_ind)
59        print("Distancia en Km: ", round(goat_apt, 3))
60
61        if Grafica:
62            plt.figure(figsize=(10,5))
63            sns.lineplot(data=df_metricas, x=df_metricas.index, y="apt", label="Best Fitness")
64            sns.lineplot(data=df_metricas, x=df_metricas.index, y="Avg_Gen_Fitness", label="Average Fitness")
65            plt.xlabel("Generation")
66            plt.ylabel("Fitness (Distance in meters)")
67            plt.title("Fitness over Generations")
68            plt.legend()
69            plt.show()
70
71    return poblacion, df_metricas, goat_ind
72
```

Figura 13: Funcion principal del algoritmo genetico

Con esta función, ejecutamos nuestro algoritmo genético para encontrar una solución aproximada al problema del vendedor viajero para lo cual usamos 4 Poblaciones iniciales diferentes con

nuestros 2 tipos de cruce y 1000 generaciones cada una, con el objetivo de comparar resultados y observar la convergencia del algoritmo. Cabe mencionar que las poblaciones iniciales fueron generadas de manera aleatoria pero se mantienen constantes para cada corrida del algoritmo de manera que nuestra comparacion sea valida.

2.9. Poblacion Inicial: 100

2.9.1. Cruce PMX

Mejor ruta: [6, 2, 7, 13, 11, 10, 15, 16, 12, 4, 1, 17, 14, 9, 3, 0, 5, 8]

Distancia en Km: 23,987.019

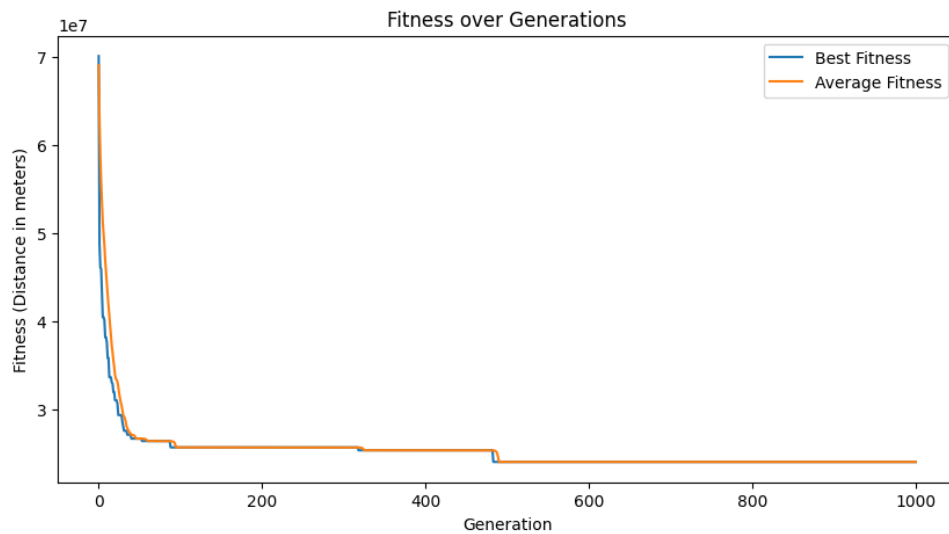


Figura 14: Evolucion de la mejor aptitud con cruce PMX y poblacion inicial 100

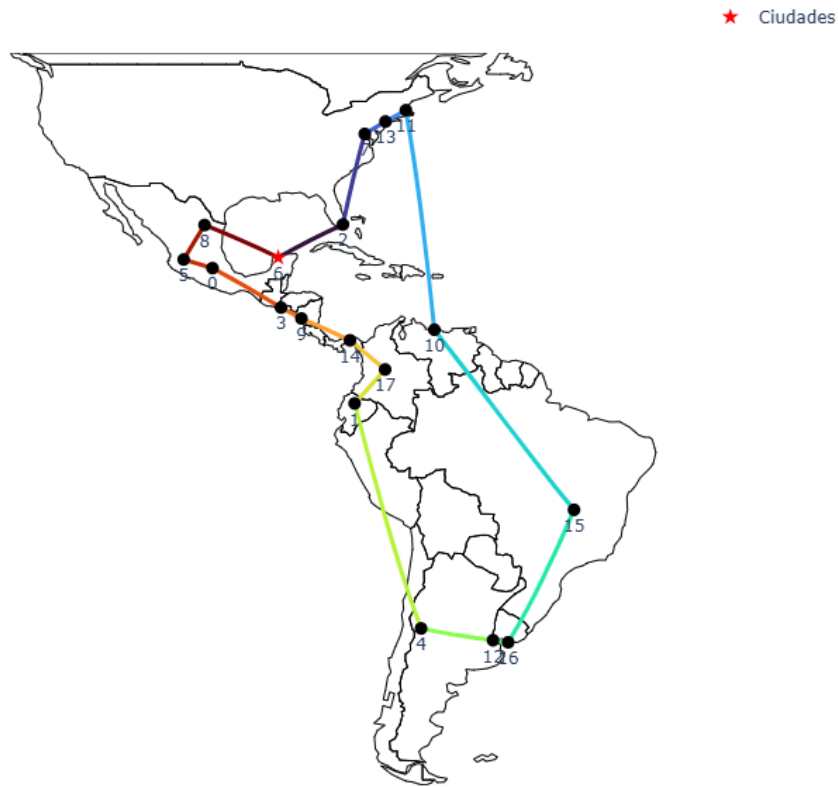


Figura 15: Mejor ruta encontrada con cruce PMX y poblacion inicial 100

2.9.2. Cruce PBX

Mejor ruta: [15, 16, 12, 4, 1, 17, 14, 9, 3, 6, 0, 5, 8, 7, 13, 11, 2, 10]

Distancia en Km: 24,052.637

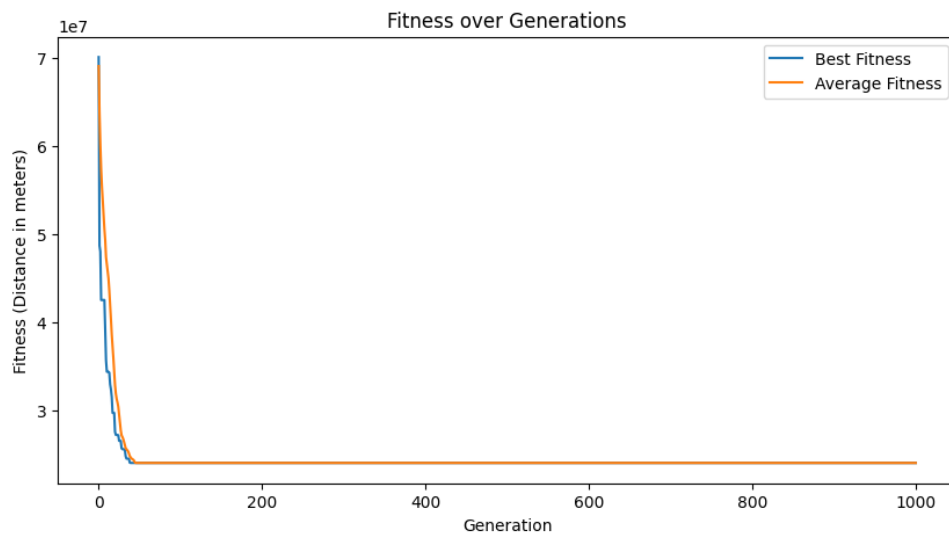


Figura 16: Evolucion de la mejor aptitud con cruce PBX y poblacion inicial 100



Figura 17: Mejor ruta encontrada con cruce PBX y poblacion inicial 100

2.10. Poblacion Inicial: 200

2.10.1. Cruce PMX

Mejor ruta: [2, 6, 8, 5, 0, 3, 9, 14, 17, 1, 4, 12, 16, 15, 10, 11, 13, 7]

Distancia en Km: 23,987.019

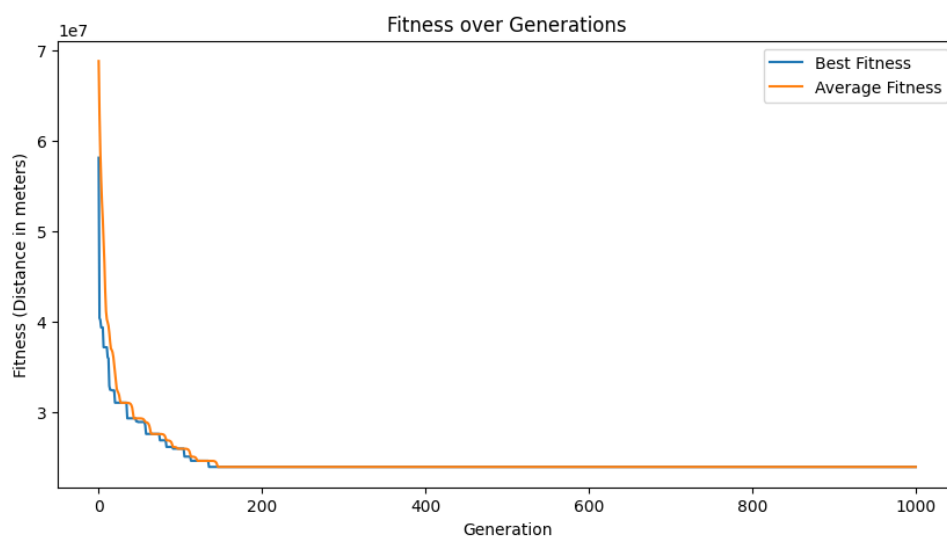


Figura 18: Evolucion de la mejor aptitud con cruce PMX y poblacion inicial 200



Figura 19: Mejor ruta encontrada con cruce PMX y poblacion inicial 200

2.10.2. Cruce PBX

Mejor ruta: [8, 5, 0, 6, 3, 9, 1, 4, 12, 16, 15, 10, 17, 14, 2, 11, 13, 7]

Distancia en Km: 24,772.018

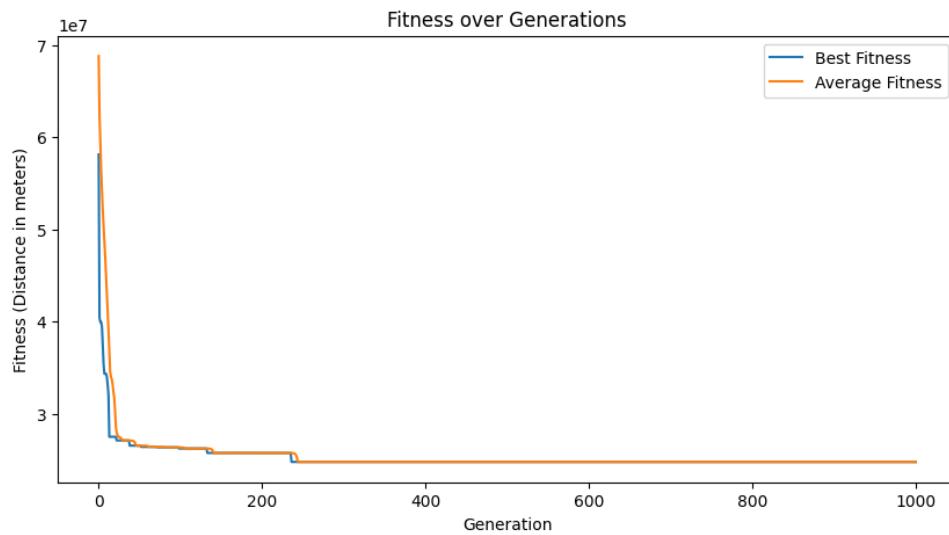


Figura 20: Evolucion de la mejor aptitud con cruce PBX y poblacion inicial 200



Figura 21: Mejor ruta encontrada con cruce PBX y poblacion inicial 200

2.11. Poblacion Inicial: 500

2.11.1. Cruce PMX

Mejor ruta: [6, 2, 7, 13, 11, 10, 15, 16, 12, 4, 1, 17, 14, 9, 3, 0, 5, 8]

Distancia en Km: 23,987.019

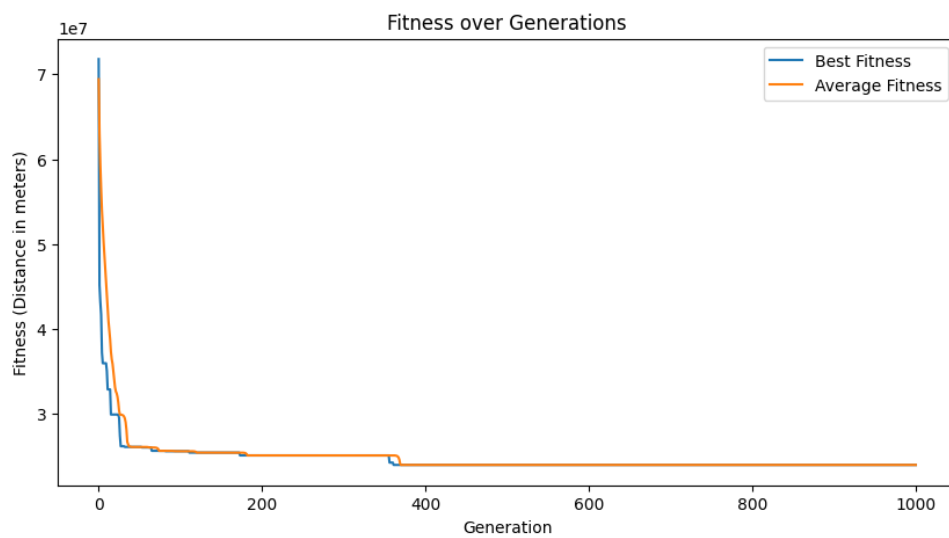


Figura 22: Evolucion de la mejor aptitud con cruce PMX y poblacion inicial 500



Figura 23: Mejor ruta encontrada con cruce PMX y poblacion inicial 500

2.11.2. Cruce PBX

Mejor ruta: [12, 16, 15, 10, 2, 11, 13, 7, 8, 5, 0, 6, 3, 9, 14, 17, 1, 4]

Distancia en Km: 24,052.637

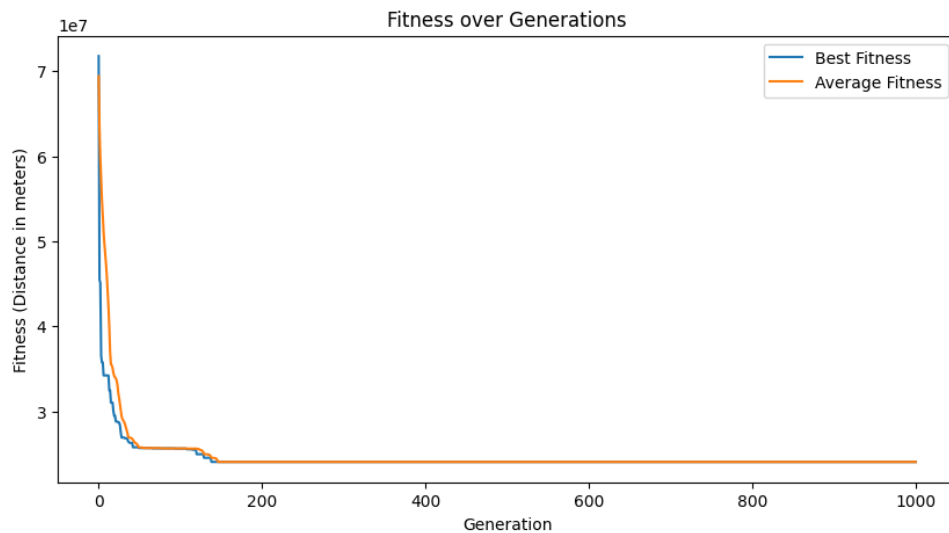


Figura 24: Evolucion de la mejor aptitud con cruce PBX y poblacion inicial 500

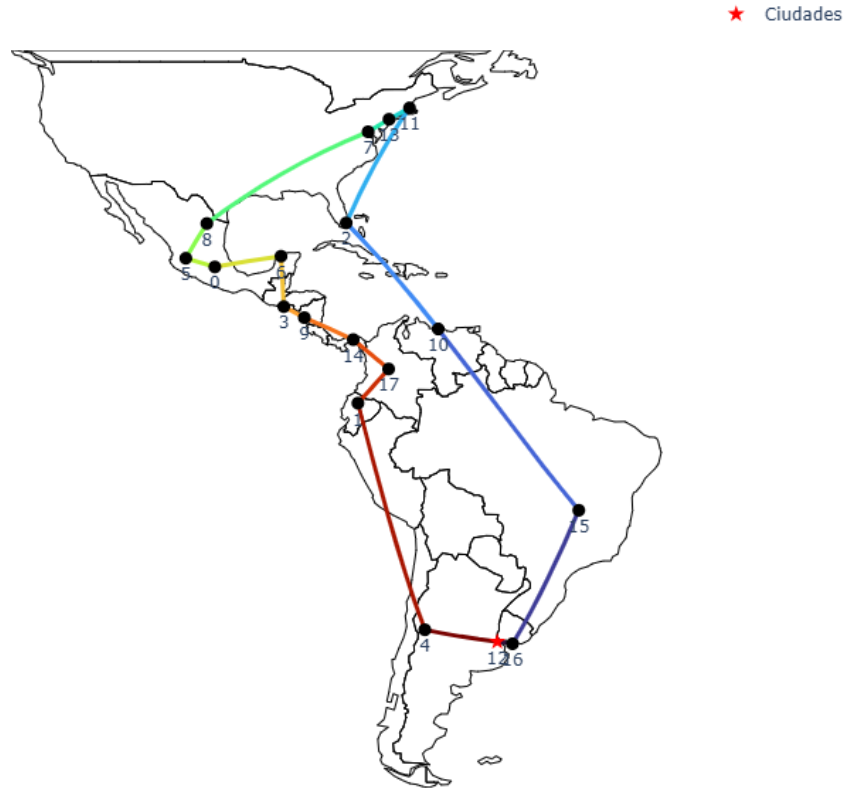


Figura 25: Mejor ruta encontrada con cruce PBX y poblacion inicial 500

2.12. Poblacion Inicial: 1000

2.12.1. Cruce PMX

Mejor ruta: [12, 16, 15, 10, 2, 11, 13, 7, 8, 5, 0, 6, 3, 9, 14, 17, 1, 4]

Distancia en Km: 24,052.637

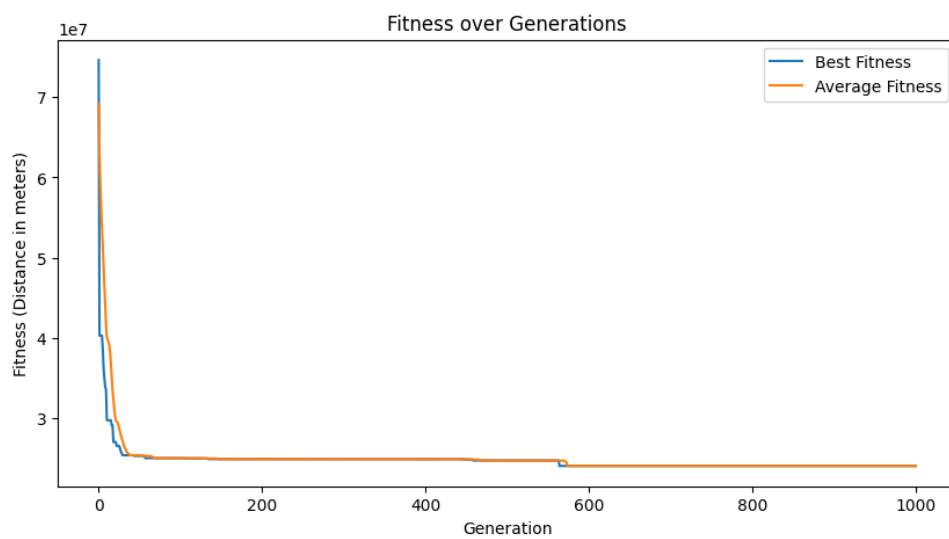


Figura 26: Evolucion de la mejor aptitud con cruce PMX y poblacion inicial 1000



Figura 27: Mejor ruta encontrada con cruce PMX y poblacion inicial 1000

2.12.2. Cruce PBX

Mejor ruta: [16, 15, 10, 17, 14, 2, 11, 13, 7, 8, 5, 0, 6, 3, 9, 1, 4, 12]

Distancia en Km: 24,772.018

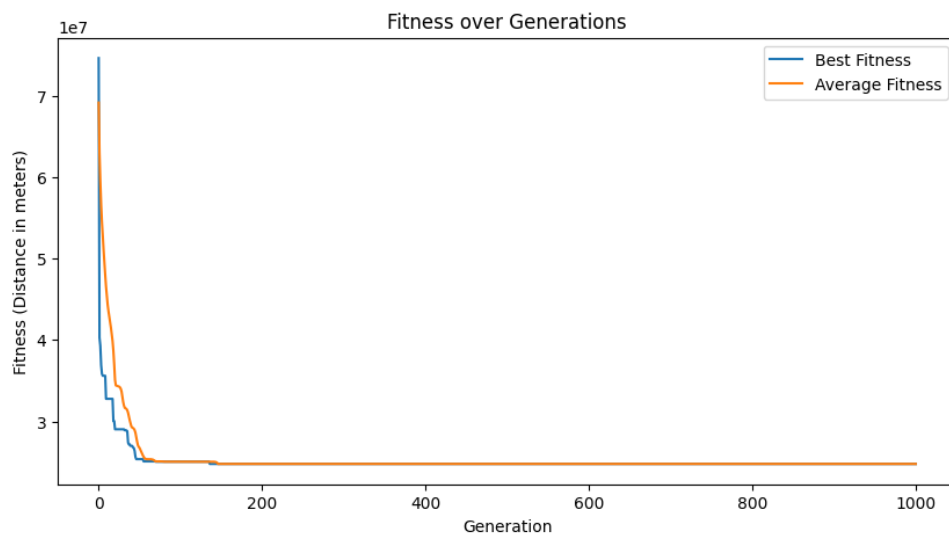


Figura 28: Evolucion de la mejor aptitud con cruce PBX y poblacion inicial 1000

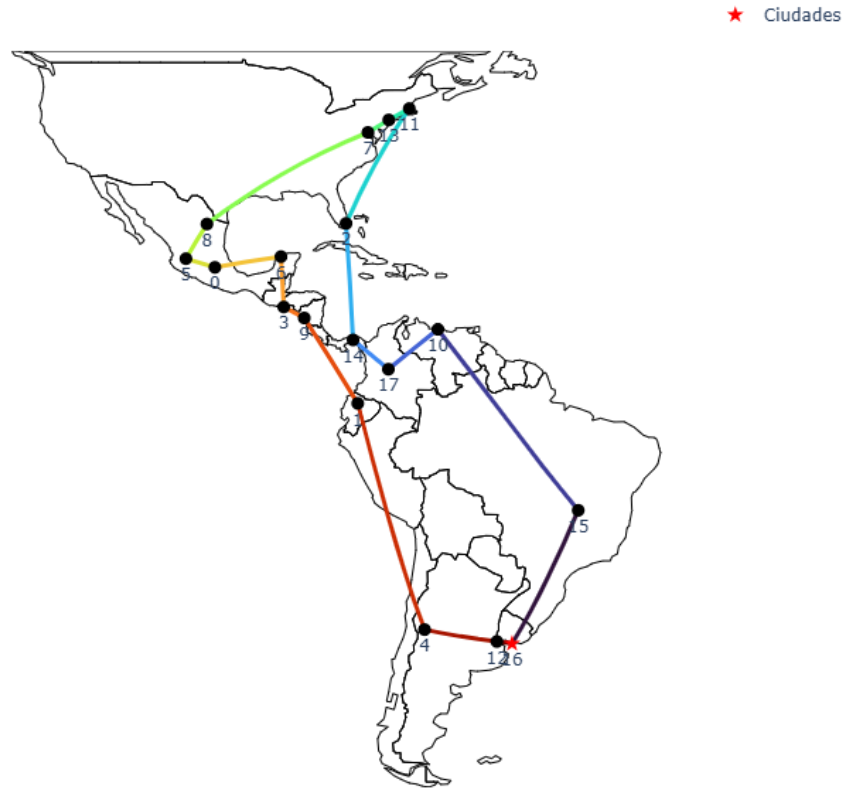


Figura 29: Mejor ruta encontrada con cruce PBX y poblacion inicial 1000

3. Resultados

Como podemos observar en las figuras 15, 19 y 23 la mejor ruta encontrada en los tres casos es la misma, lo que indica que el algoritmo es capaz de converger a una solución óptima o cercana a la óptima independientemente del tamaño de la población inicial. Además, se puede notar que a medida que aumenta el tamaño de la población, la diversidad de soluciones también aumenta, lo que puede ayudar a evitar caer en óptimos locales.

En los 3 casos mencionados la mejor aptitud o bien la distancia más corta encontrada es de 23,987.019 km, lo que representa una mejora significativa en comparación con la ruta inicial aleatoria. Esto demuestra la efectividad del algoritmo genético para optimizar rutas en el problema del vendedor viajero.

Por otro lado las 3 soluciones se presentan con la cruza PMX lo que indica que este método de cruza es más efectivo en este caso específico en comparación con PBX, sin embargo, no se puede descartar que los resultados obtenidos con PBX son bastante cercanos y también muestran un desempeño aceptable y reproducible. Podemos ver tambien las otras rutas bastante aceptables tambien que se repiten las cuales son:

Distancia de 24,052.637 km

- pbx 100 figura 17
- pbx 500 figura 25
- pmx 1000 figura 27

Distancia de 24,772.018 km

- pbx 200 figura 21
- pbx 1000 figura 29

También en las graficas de aptitud vs generaciones podemos observar que en todos los casos la aptitud mejora rápidamente en las primeras generaciones y luego se estabiliza, lo que indica que el algoritmo está convergiendo hacia una solución óptima o cercana a la óptima.

4. Discusión

Como se mencionó en la sección 1.4, el cálculo de distancias entre ciudades se realizó utilizando la fórmula de Haversine, que es adecuada para calcular distancias entre puntos en la superficie de una esfera, como la Tierra. Esta elección es crucial para obtener resultados precisos en el contexto del TSP, ya que las distancias reales entre ciudades afectan directamente la optimización de la ruta. Este es el primer punto a considerar al analizar los resultados obtenidos ya que el calculo con coordenadas planas hubiera introducido errores significativos en la estimación de distancias reales entre ciudades ya que se consideraria una linea recta que en una esfera corta partes de la superficie, estas ligeras curvaturas pueden verse en los mapas de las rutas obtenidas.

Para la parte de la cruce, se utilizaron dos métodos: PMX y PBX, ambos diseñados para preservar la validez de las rutas generadas el principal componente en el PMX durante la implementacion fue asegurar que no se repitieran ciudades en la ruta hija, lo cual se logró mediante un mapeo cuidadoso de los segmentos intercambiados entre los padres ya que en primeras instancias se presentaron errores donde se hacia un ciclo infinito por que no existian posiciones validas para las ciudades, por lo cual se implementaron validaciones adicionales para garantizar que cada ciudad apareciera exactamente una vez en la ruta resultante. En el caso del PBX, el desafío principal fue mantener el orden relativo de las ciudades no seleccionadas, lo cual se logró mediante la eliminación cuidadosa de duplicados y el llenado de las posiciones restantes con los elementos sobrantes del segundo padre.

En cuanto a los métodos de mutación, la mutación por barajado (scramble) y la mutación por intercambio de bloques (block transpose) fueron seleccionadas por su capacidad para introducir diversidad en la población sin alterar significativamente la estructura de las rutas. La mutación por barajado altera el orden interno de un subconjunto de ciudades, lo que puede ayudar a explorar nuevas combinaciones sin cambiar la posición relativa de las ciudades fuera del subconjunto afectado. Por otro lado, la mutación por intercambio de bloques modifica la posición de segmentos enteros de la ruta, lo que puede resultar en cambios más drásticos en la estructura de la solución, pero también en una mayor exploración del espacio de búsqueda.

Con respecto a los resultados obtenidos, es evidente que el tamaño de la población inicial tiene un impacto significativo en la diversidad de soluciones exploradas por el algoritmo. Una población más grande tiende a ofrecer una mayor variedad de rutas, lo que puede ayudar a evitar la convergencia prematura a óptimos locales. Sin embargo, también implica un mayor costo computacional, ya que se deben evaluar más soluciones en cada generación. También podemos observar que la cruce PBX tiende a generar soluciones ligeramente menos óptimas pero más rapidas al momento de converger en comparación con PMX en este caso específico

pero este ultimo presenta mejores resultados, aunque ambas cruces demostraron ser efectivas para explorar el espacio de soluciones.

Podemos observar que tambien el tener tantas generaciones (1000) puede ser contraproducente ya que en la mayoria de los casos la aptitud se estabiliza mucho antes de llegar a este punto, por lo que se podria considerar reducir el numero de generaciones para mejorar la eficiencia del algoritmo sin sacrificar significativamente o principalmente buscar un mejor criterio de paro que no sea un numero fijo de generaciones.

5. Conclusión

En conclusión, la implementación de algoritmos genéticos para resolver el Problema del Vendedor Viajero (TSP) ha demostrado ser una estrategia efectiva para encontrar soluciones cercanas a la óptima en un espacio de búsqueda complejo. La elección de métodos de cruce como PMX y PBX, junto con técnicas de mutación como scramble y block transpose, ha permitido mantener la diversidad genética en la población y explorar una amplia gama de soluciones posibles.

El análisis de los resultados indica que el tamaño de la población inicial y el número de generaciones juegan un papel crucial en la calidad de las soluciones obtenidas. Una población más grande tiende a ofrecer una mayor variedad de rutas sin embargo podemos ver que no es un criterio suficiente para garantizar mejores resultados, ya que la aptitud se estabiliza después de un cierto número de generaciones, sugiriendo que la convergencia puede ocurrir rápidamente en algunos casos. Esto resalta la importancia de considerar criterios de paro más dinámicos que un número fijo de generaciones, como la mejora en la aptitud o la diversidad de la población.

Este problema tiene múltiples aplicaciones de manera directa y práctica en la vida real, lo que aporta un valor adicional a la correcta implementación y análisis de estos algoritmos. En futuras investigaciones, sería interesante explorar la combinación de estos métodos con otras técnicas de optimización para comparar resultados.

Una de las aplicaciones fuera de las más comunes sería justamente la aplicación inversa de los resultados, es decir dado un conjunto de puntos y rutas determinar la ubicación más lejana, esto podría usarse en temas de seguridad, por ejemplo para determinar la mejor ubicación de una base militar o de policía, o bien para estudios de mercado y ubicación de tiendas o centros comerciales, entre otras aplicaciones.

6. Bibliografía

Referencias

- [1] Sanjeev Arora. «Polynomial time approximation schemes for Euclidean traveling salesman and other geometric problems». En: *Journal of the ACM* 45.5 (1998), págs. 753-782. DOI: 10.1145/290179.290180.
- [2] Tolga Bektas. «The multiple traveling salesman problem: an overview of formulations and solution procedures». En: *Omega* 34.3 (2006), págs. 209-219. DOI: 10.1016/j.omega.2004.08.001.